

Station Audio Toolkit — Implementation Specification

Document Purpose

This is the complete implementation specification for a Pure Data (Pd) interactive audio toolkit designed to produce broadcast-quality audio elements for a non-commercial educational radio station. A developer (Claude Code) should be able to build the entire toolkit from this document alone.

The toolkit produces jingles, sweepers, stingers, bumpers, beds, and other short-form audio elements that give the station its sonic identity. The design philosophy prioritizes immediate usability: every module makes sound on first open, every default is opinionated and broadcast-ready, and every control is labeled for a creative who may not know synthesis.

Environment and Dependencies

Required Software

- **Pure Data vanilla** — current stable release (0.54+). Download from <https://puredata.info>
- **ELSE library** — installed via Pd's built-in Deken package manager (Help > Find Externals > search "else"). This is the only external dependency. ELSE provides band-limited oscillators, superior filters, freeverb, and utility objects that vanilla lacks.
- **espeak-ng** — for any text-to-speech needs in the post-session pipeline (already in the station's automation stack)
- **ffmpeg** — for post-session loudness normalization, MP3 encoding, and ID3 tagging (already in the station's automation stack)

Optional

- **bob~** — Moog ladder filter emulation, included in Pd vanilla since version 0.51. Patches should check for availability and use it when present, falling back to ELSE's `svfilter~` if not. No separate install needed.

What NOT to Install

- Do not use Pd-extended (abandoned since 2014)
 - Do not use Purr Data (smaller community, less predictable updates)
 - Do not install cyclone, zexy, Gem, ofelia, or pd-lua — they are unnecessary for this project
-

Architecture

Deliverables

The toolkit consists of:

1. A set of Pure Data patch files (`.pd`), each one a module
2. A main console patch that ties them together
3. A post-session shell script for normalization, MP3 conversion, and ID3 tagging
4. A README with setup and usage instructions

File Structure

```
station-audio-toolkit/
├── README.md
├── console.pd      — Main console (always open)
├── modules/
│   ├── osc-bank.pd  — Analog Oscillator Bank
│   ├── noise-sculptor.pd — Noise Sculptor
│   ├── fm-voice.pd  — FM Voice
│   ├── chord-pad.pd  — Chord Pad
│   ├── sample-player.pd — Sample Player
│   ├── sequencer.pd  — Step Sequencer
│   ├── tape-warmth.pd — Tape Warmth (inline effect)
│   ├── space.pd      — Room/Reverb (inline effect)
│   ├── chorus.pd     — Chorus/Ensemble (send effect)
│   ├── filter.pd     — Standalone Filter (send effect)
│   └── tremolo-gate.pd — Tremolo/Gate (send effect)
├── presets/
│   └── (preset value files, one per module, loaded on init)
├── export/
│   ├── raw/          — Original WAV files from sessions
│   ├── jingles/
│   ├── sweepers/
│   ├── stingers/
│   ├── bumpers/
│   ├── beds/
│   ├── station-id/
│   ├── psa/
│   ├── promos/
│   └── other/
```

└─ scripts/
└─ post-session.sh — Normalization, MP3 conversion, ID3 tagging

Signal Flow

The master signal flow is hardwired in the console patch:



- The **insert bus** is a stereo summing mixer. Each active source module feeds it via a per-source level and pan control.
- **Tape Warmth** and **Space** are inline on the master chain — active by default with subtle settings. They define the "station sound."
- The **send bus** is parallel. Each source module has a send knob that taps signal to the send bus. The send bus feeds Chorus, Filter, and Tremolo in parallel (not series). Each send effect has its own return level that feeds back into the master bus.
- The **sequencer** is a control-only module. It sends note and gate messages to a target source module. No audio passes through it.
- **MIDI input** routes to a selectable source module via the console's MIDI routing panel.

Module Specifications

Each module below is a separate `.pd` file. All modules must:

- Make sound (or be ready to route control data) immediately upon being toggled on
- Ship with the specified default parameter values already set
- Include labeled preset buttons that snap all parameters to the specified preset values
- Have every control labeled with human-readable text (not abbreviated Pd object names)
- Constrain slider/control ranges to the specified sweet-spot ranges
- Display current parameter values numerically beside each control
- Include a brief plain-text comment at the top of the patch explaining what the module does

GUI Conventions

- Use Pd canvas objects (colored rectangles) as background panels for visual grouping

- Place a title banner (large text label) at the top of every module
 - Group related controls visually (e.g., all envelope controls together)
 - Use consistent sizing: sliders should be long enough for fine control (at least 128px)
 - Number boxes beside sliders should show the current value
 - Toggle and radio-button selections should have text labels for each option
 - Color scheme: use a consistent neutral background with contrasting label text
-

Source Modules

1. Analog Oscillator Bank (osc-bank.pd)

Two oscillators with mix, shared filter, and amplitude envelope.

Description text for patch: "Two-oscillator analog-style synthesizer. Sawtooth, square, triangle, and sine waveforms with independent pitch, detune, and drift. Shared resonant filter and ADSR envelope. The workhorse for tonal content — leads, basses, and melodic lines."

Signal chain: Osc1 + Osc2 (mixed) → Filter → Amplitude Envelope → Module Output

Oscillator implementation: Use ELSE band-limited oscillators (bl.saw~, bl.square~, bl.tri~) to avoid aliasing. Use vanilla osc~ for sine. Each oscillator has an independent waveform selector.

Filter implementation: Use bob~ if available (Moog ladder emulation — check with [declare-lib bob~] or similar). Fall back to ELSE svfilter~ (lowpass output) if bob~ is not present.

Drift implementation: Each oscillator has an independent slow LFO (use osc~ at sub-audio rate) routed to pitch. The LFO rates should differ between oscillators so their drift patterns never synchronize.

Default State (what plays on first open)

Parameter	Oscillator 1	Oscillator 2
Waveform	Band-limited sawtooth	Band-limited sawtooth
Pitch	C2 (65.4 Hz)	C2 (65.4 Hz)
Fine tune	0 cents	+7 cents
Drift amount	3 cents	3 cents
Drift LFO rate	0.1 Hz	0.13 Hz

Parameter	Value
Osc mix	50/50
Filter type	Low-pass
Filter cutoff	2200 Hz
Filter resonance	15%
Envelope attack	20 ms
Envelope decay	200 ms
Envelope sustain	80%
Envelope release	300 ms
Module output level	-6 dB

Control Ranges

Control	Min	Max	Default	Display
Pitch (per osc)	C0	C5	C2	Note name
Fine tune (per osc)	-50 cents	+50 cents	0 / +7	Cents
Drift amount (per osc)	0 cents	10 cents	3	Cents
Waveform (per osc)	—	—	Saw	Radio: Saw / Square / Tri / Sine
Osc mix	0% (osc1 only)	100% (osc2 only)	50%	Percent
Filter cutoff	60 Hz	6000 Hz	2200 Hz	Hz
Filter resonance	0%	70%	15%	Percent
Attack	1 ms	500 ms	20 ms	ms
Decay	10 ms	2000 ms	200 ms	ms
Sustain	0%	100%	80%	Percent
Release	10 ms	3000 ms	300 ms	ms

Presets

"Warm Bass" Both oscillators: sawtooth, C2, osc2 detune +7, filter cutoff 800 Hz, resonance 20%, attack 20 ms, decay 200 ms, sustain 80%, release 300 ms.

"Bright Lead" Osc1: sawtooth, C3. Osc2: square, C3, detune +5. Filter cutoff 4000 Hz, resonance 25%, attack 15 ms, decay 150 ms, sustain 70%, release 250 ms.

"Sub Drone" Both oscillators: sine (osc1) and triangle (osc2), C1, detune 0, drift 5 cents. Filter cutoff 6000 Hz (wide open), resonance 5%, attack 50 ms, decay 500 ms, sustain 100%, release 500 ms.

"Hollow Wood" Osc1: square, C3. Osc2: triangle, C3, detune +12. Filter cutoff 1500 Hz, resonance 35%, attack 5 ms, decay 300 ms, sustain 50%, release 200 ms.

"Analog Strings" Both oscillators: sawtooth, C3, detune +15. Filter cutoff 3000 Hz, resonance 10%, attack 200 ms, decay 400 ms, sustain 90%, release 800 ms. Drift 4 cents.

Input

Responds to note (pitch + gate) messages from the sequencer or MIDI input. Gate on triggers the envelope attack. Gate off triggers the release. Pitch sets the base frequency of both oscillators (fine tune and detune are relative to this).

2. Noise Sculptor (`noise-sculptor.pd`)

Noise source through a resonant filter with LFO modulation and optional envelope shaping.

Description text for patch: "Colored noise through a resonant filter with slow modulation. Produces textures, beds, whooshes, and transition effects. Continuous mode runs freely for ambient layers. Shape mode triggers swells and fades for one-shot effects."

Signal chain: Noise Source → Filter (with LFO modulation) → Amplitude Shaper → Module Output

Noise implementation: Use vanilla `noise~` for white noise. Derive pink noise by filtering white through `lop~` at appropriate slope, or use ELSE `pink~` if available. Derive brown noise with a leaky integrator (single-sample feedback loop with a coefficient around 0.99, then normalize).

Filter implementation: Use ELSE `svfilter~` for multimode (LP/BP/HP from a single object).

Two modes:

- **Continuous** (default): noise runs constantly, shaped only by filter and LFO. No envelope.
- **Shape:** an AHR (Attack-Hold-Release) envelope is triggered by sequencer gate, MIDI note, or a manual trigger button in the GUI. In shape mode, the filter cutoff can also be envelope-modulated (cutoff sweeps upward during attack for risers).

Default State

Parameter	Value
Noise color	Pink
Filter type	Bandpass
Filter center frequency	400 Hz
Filter resonance	25%
LFO rate	0.05 Hz
LFO depth	30%
Mode	Continuous
Shape attack	500 ms
Shape hold	200 ms
Shape release	1000 ms
Module output level	-6 dB

Control Ranges

Control	Min	Max	Default	Display
Noise color	—	—	Pink	Radio: White / Pink / Brown
Filter type	—	—	BP	Radio: LP / BP / HP
Filter center	40 Hz	8000 Hz	400 Hz	Hz
Resonance	0%	75%	25%	Percent
LFO rate	0.01 Hz	2 Hz	0.05 Hz	Hz
LFO depth	0%	100%	30%	Percent
Mode	—	—	Continuous	Radio: Continuous / Shape
Attack	10 ms	3000 ms	500 ms	ms
Hold	0 ms	2000 ms	200 ms	ms
Release	50 ms	5000 ms	1000 ms	ms

Presets

"Warm Breath" The default state. Pink noise, BP at 400 Hz, gentle LFO.

"Vinyl Surface" Two noise layers mixed internally: brown noise (low rumble, LP at 200 Hz, level -12 dB) and white noise (high crackle, HP at 3000 Hz, level -18 dB). No LFO. Simulates a record playing.

"Ocean Wash" Brown noise, LP at 600 Hz, LFO rate 0.03 Hz, depth 80% sweeping cutoff from 200 Hz to 1200 Hz. Deep and rhythmic.

"Rising Sweep" White noise, shape mode, BP starting at 200 Hz. Envelope attack 2000 ms, hold 0 ms, release 500 ms. Filter cutoff is envelope-modulated: sweeps from 200 Hz to 4000 Hz during attack phase. Trigger produces a rising whoosh.

"Soft Static" White noise, LP at 1500 Hz, no LFO, output level -18 dB. A subtle textural layer meant to sit underneath tonal content.

3. FM Voice (**fm-voice.pd**)

Two-operator FM synthesizer. Carrier oscillator frequency-modulated by a modulator oscillator.

Description text for patch: "Two-operator FM synthesizer. Creates bell tones, metallic hits, warm keys, and evolving textures. The Character control sets the harmonic relationship. The Brightness control sets timbral

complexity. Great for station ident hits and percussive elements."

Signal chain: Modulator Osc → (× Index) → Carrier Osc FM input → Amplitude Envelope → Module Output

Implementation: Use vanilla `osc~` for both carrier and modulator. FM synthesis in Pd is straightforward: modulator output scaled by index is added to carrier frequency input. The modulator ratio is expressed as a multiplier of the carrier frequency.

Key design detail: The modulator ratio selector should snap to discrete values with human-readable labels, not be continuously variable. This keeps the creative in musically useful territory.

Default State

Parameter	Value
Carrier pitch	C4 (261 Hz)
Mod ratio	2:1 (labeled "Octave")
Mod index (Brightness)	1.5
Brightness envelope amount	1.5 (added to base index at note onset)
Brightness envelope decay	400 ms
Amplitude attack	5 ms
Amplitude decay	600 ms
Amplitude sustain	40%
Amplitude release	400 ms
Module output level	-6 dB

Control Ranges

Control	Min	Max	Default	Display
Carrier pitch	C1	C6	C4	Note name
Character (mod ratio)	—	—	2:1	Radio with labels (see below)
Brightness (mod index)	0	8	1.5	Number
Brightness env amount	0	10	1.5	Number
Brightness env decay	20 ms	2000 ms	400 ms	ms
Amp attack	1 ms	500 ms	5 ms	ms
Amp decay	10 ms	2000 ms	600 ms	ms
Amp sustain	0%	100%	40%	Percent
Amp release	10 ms	3000 ms	400 ms	ms

Modulator Ratio Options (Character Selector)

Ratio	Label	Sound Character
0.5	Sub-octave	Warm, sub-harmonic undertone
1	Unison	Thickened fundamental, organ-like
1.5	Fifth-ish	Slightly inharmonic, bell-adjacent
2	Octave	Clean bell tones (default)
3	Fifth+Oct	Brighter bell, more overtones
4	Two Octaves	Bright, crystalline
5	Bright	Complex, shimmering
7	Metallic	Inharmonic, percussive

Presets

"Bell Tone" The default state. Clean warm bell at C4, ratio 2:1, moderate brightness with envelope.

"**Glass Chime**" Carrier C5, ratio 3:1, index 2.0, index envelope amount 4.0, envelope decay 150 ms. Bright, crystalline, short.

"**Warm Keys**" Carrier C3, ratio 1:1, index 0.8, minimal index envelope (amount 0.3). Mellow, electric piano feel.

"**Metallic Ping**" Carrier C4, ratio 7:1, index 2.5, index envelope amount 5.0, decay 100 ms. Sharp inharmonic percussive hit.

"**Deep Gong**" Carrier C2, ratio 1.5:1, index 3.0, index envelope amount 2.0, very slow decay 2000 ms. Amplitude release 3000 ms. Low, evolving, dramatic.

Input

Responds to note (pitch + gate) messages from sequencer or MIDI. Pitch sets carrier frequency. Gate triggers both amplitude and brightness envelopes.

4. Chord Pad (`chord-pad.pd`)

Multiple detuned oscillators tuned to a chord. Instant bed material.

Description text for patch: "Sustained chord tones with shimmer and movement. Select a root note and chord type, and the module voices multiple detuned oscillators to create a rich, wide pad sound. The primary tool for creating beds that sit under speech."

Signal chain: Multiple Oscillator Voices (tuned to chord intervals) → Summed → Filter (with LFO) → Amplitude Envelope → Module Output

Implementation: For each pitch in the chord, instantiate the specified number of voices (2 or 4 band-limited sawtooths via ELSE `bl.saw~`). Detune voices symmetrically around the target pitch (e.g., with 4 voices and 10 cents spread: -5, -2, +2, +5 cents). Sum all voices, then filter and envelope.

Chord voicing: Given a root note and chord type, calculate the MIDI note numbers for each pitch in the chord. Chords are voiced in a single octave from the root.

Chord Types

Type	Label	Intervals (semitones from root)
open5	Open Fifth	0, 7
major	Major	0, 4, 7
minor	Minor	0, 3, 7
sus2	Sus2	0, 2, 7
sus4	Sus4	0, 5, 7
maj7	Major 7th	0, 4, 7, 11
min7	Minor 7th	0, 3, 7, 10
oct	Octaves	0, 12

Default State

Parameter	Value
Root note	G2
Chord type	Open Fifth
Voices per pitch	4
Detune spread	10 cents
Filter cutoff	1800 Hz
Filter resonance	10%
Filter LFO rate	0.03 Hz
Filter LFO depth	25%
Amplitude attack	800 ms
Amplitude release	1500 ms
Module output level	-6 dB

Control Ranges

Control	Min	Max	Default	Display
Root note	C1	C4	G2	Note name
Chord type	—	—	Open Fifth	Radio selector
Voices per pitch	—	—	4	Radio: 2 / 4
Detune spread	0 cents	30 cents	10 cents	Cents
Filter cutoff	100 Hz	4000 Hz	1800 Hz	Hz
Filter resonance	0%	40%	10%	Percent
Filter LFO rate	0.01 Hz	0.5 Hz	0.03 Hz	Hz
Filter LFO depth	0%	60%	25%	Percent
Attack	50 ms	3000 ms	800 ms	ms
Release	100 ms	5000 ms	1500 ms	ms

Presets

- "**Warm Bed**" The default state. G2 open fifth, slow and warm.
- "**Morning Shimmer**" Root C3, Major 7th, detune 15 cents, filter cutoff 3000 Hz, LFO rate 0.1 Hz. Bright, optimistic, more movement.
- "**Deep Drone**" Root C1, Open Fifth, 4 voices, detune 5 cents, filter cutoff 600 Hz, no LFO (depth 0%). Dark, low, minimal.
- "**Sunset Glow**" Root D2, Sus2, detune 12 cents, filter cutoff 2500 Hz, LFO rate 0.02 Hz. Open, wistful, peaceful.
- "**Jazz Murmur**" Root Eb2, Minor 7th, filter cutoff 1200 Hz, resonance 20%. Warm, slightly smoky.

Input

The chord pad is primarily a sustained drone — it turns on and stays on. It does not need note-by-note triggering from the sequencer. However, it should accept a root note change from MIDI (incoming MIDI note sets the root, chord type is preserved). The amplitude envelope triggers when the module is toggled on and releases when toggled off.

5. Sample Player (sample-player.pd)

Load and play back WAV files with manipulation controls.

Description text for patch: "Load any WAV file and play it back with speed, pitch, and loop controls. Use for field recordings, vocal snippets, spoken legal IDs, or found sounds. Integrates real-world audio into the toolkit's signal chain and effects."

Signal chain: File Playback → Speed/Pitch Control → Loop Engine → Level Control → Module Output

Implementation: Use vanilla `readsf~` for basic playback. For variable-speed playback with pitch change, use `tabplay~` or `tabread4~` reading from an array at variable rate. Load files into a Pd array using `soundfiler`. Display the filename and duration after loading.

Default State

Parameter	Value
No sample loaded	(controls at neutral)
Playback speed	1.0x
Loop	Off
Loop start	0%
Loop end	100%
Direction	Forward
Output level	-6 dB

Control Ranges

Control	Min	Max	Default	Display
Speed	0.25x	2.0x	1.0x	Multiplier
Loop	—	—	Off	Toggle: On / Off
Loop start	0%	100%	0%	Percent of file
Loop end	0%	100%	100%	Percent of file
Direction	—	—	Forward	Radio: Forward / Reverse
Level	-inf	0 dB	-6 dB	dB

Controls

- **Load button** (bang) — opens file dialog to select a WAV file
- **Play / Stop** toggle
- **Filename display** — shows loaded file name and duration
- All controls listed above

Input

Play/stop can be triggered by sequencer gate (gate on = play, gate off = stop). This allows sequenced triggering of samples.

6. Step Sequencer (`sequencer.pd`)

Control-signal-only module. Generates note and gate data to drive source modules.

Description text for patch: "Step sequencer for creating melodic and rhythmic patterns. Drives any source module with note and gate data. Scale constraint keeps everything musical — pentatonic minor is nearly impossible to make sound bad. Start with a preset pattern and tweak from there."

Implementation: Use a `metro` object clocked by BPM. A step counter (`mod` of step count) advances through arrays storing pitch, gate, and accent values per step. Output note and gate messages routed to the selected target module. Visual step indicators (toggle or canvas objects) light up to show the current step.

Scale constraint: The per-step pitch selectors should only offer notes that belong to the currently selected scale and root. When the scale or root changes, all existing step pitches should snap to the nearest note in the new scale.

Scale Definitions (intervals from root, in semitones)

Scale	Intervals
Chromatic	0,1,2,3,4,5,6,7,8,9,10,11
Major	0,2,4,5,7,9,11
Minor	0,2,3,5,7,8,10
Pentatonic Major	0,2,4,7,9
Pentatonic Minor	0,3,5,7,10
Blues	0,3,5,6,7,10

Notes available span from 2 octaves below the root to 2 octaves above (5 octaves total range for the step pitch selectors).

Default State

Parameter	Value
Step count	8
Tempo	92 BPM
Gate length	70%
Swing	0%
Direction	Forward
Scale	Pentatonic Minor
Root	G
Target	(first active source module)
Status	Stopped

Default pattern (8 steps):

Step	1	2	3	4	5	6	7	8
Pitch	G3	F3	E3	D3	C3	D3	E3	F3
Gate	On	On	On	On	On	On	On	On
Accent	Normal	Normal	Normal	Normal	Normal	Normal	Normal	Normal

Note: this default pattern uses a minor scale. When the scale constraint is set to pentatonic minor (default), these pitches should snap to nearest: G3, D3, C3, D3, C3, D3, D3, D3 (or nearest pentatonic equivalent).
Alternatively, define the default pattern natively in G pentatonic minor: G3, D3, C3, A2, G2, A2, C3, D3. This is the preferred approach — a descending-then-ascending pentatonic line.

Control Ranges

Control	Min	Max	Default	Display
Step count	—	—	8	Radio: 4 / 8 / 12 / 16
Tempo	40 BPM	160 BPM	92 BPM	BPM
Gate length	10%	100%	70%	Percent
Swing	0%	60%	0%	Percent
Direction	—	—	Forward	Radio: Forward / Reverse / Pendulum / Random
Scale	—	—	Pent. Minor	Radio selector
Root note	—	—	G	Radio: C through B
Target module	—	—	(first active)	Radio selector

Per-step controls (repeated for each step):

- Pitch: selector constrained to current scale, displayed as note name
- Gate: on/off toggle
- Accent: normal/accent toggle

Transport:

- Run / Stop toggle (large, obvious)
- Current step indicator (visual highlight on active step)

Presets

"**Gentle Descent**" The default pattern. G pentatonic minor descending and ascending: G3, D3, C3, A2, G2, A2, C3, D3.

"**Rhythmic Pulse**" Single note (root G3), all 8 steps active, accents on steps 1, 3, 5, 7. Purely rhythmic.

"**Call and Response**" 8 steps: G3 (on), A3 (on), C4 (on), rest (off), D3 (on), C3 (on), rest (off), rest (off). Phrased, conversational.

"**Slow Arp**" 4 steps only, tempo 60 BPM: G2, Bb2, D3, G3. Ascending, spacious.

"**Random Walk**" Direction set to random, all 8 steps active with varied pitches across the scale range. Every pass different, always harmonically safe.

Effects Modules

7. Tape Warmth (`tape-warmth.pd`)

Inline on master chain. Defines the station's sonic character.

Description text for patch: "Tape emulation with saturation, bass emphasis, and subtle pitch flutter. This module is the core of the station sound — it sits inline on the master chain and gently warms everything that passes through. Defaults are subtle. You should feel the difference when you bypass it, not hear the effect while it's active."

Signal chain (internal): Input → Saturation Stage → Head Bump EQ → Flutter → Output

Saturation implementation: Waveshaping using hyperbolic tangent ($\tanh$). In Pd, this can be approximated with `expr~ tanh($v1 * drive)` or using ELSE's `drive~` if available. The drive parameter scales the input before the waveshaper; the output is then normalized to prevent level jumps.

Head bump implementation: A gentle bell EQ boost at 100 Hz. Implement with a bandpass filter at 100 Hz mixed in parallel with the dry signal at a controlled level.

Flutter implementation: A slow LFO modulating a very short delay line (< 1ms variation). This produces subtle pitch modulation without audible delay.

CRITICAL DESIGN PRINCIPLE: The default must be barely perceptible. The creative should only notice it by toggling bypass.

Default State

Parameter	Value
Drive	15%
Tone (tilt EQ)	+1 dB (toward warm)
Head bump	+1.5 dB at 100 Hz
Flutter rate	2 Hz
Flutter depth	2 cents
Bypass	OFF (effect active)
Mix	100%

Control Ranges

Control	Min	Max	Default	Display
Drive	0%	60%	15%	Percent
Tone	-6 dB	+6 dB	+1 dB	dB (negative=bright, positive=warm)
Head bump	0 dB	+4 dB	+1.5 dB	dB
Flutter rate	0.5 Hz	6 Hz	2 Hz	Hz
Flutter depth	0 cents	15 cents	2 cents	Cents
Bypass	—	—	Off	Toggle
Mix	0%	100%	100%	Percent

No presets. The default state IS the preset. Include an A/B bypass toggle for quick comparison.

8. Space (space.pd)

Inline on master chain after tape warmth. Room reverb.

Description text for patch: "Room reverb that places all sounds in a physical space. Active by default with subtle settings — everything sounds like it's in a real room rather than a digital void. Push the size and wet controls for more dramatic effects."

Implementation: Use ELSE freeverb~. Single object, stereo in/out, well-tuned algorithm.

Default State

Parameter	Value
Size	30%
Damping	60%
Width	80%
Dry/Wet	12%
Bypass	OFF (effect active)

Control Ranges

Control	Min	Max	Default	Display
Size	0%	80%	30%	Percent
Damping	0%	100%	60%	Percent
Width	0%	100%	80%	Percent
Dry/Wet	0%	50%	12%	Percent
Bypass	—	—	Off	Toggle

Note: Dry/Wet capped at 50% because this is an insert effect. At 50% the reverb tail is equal to the dry signal, which is the maximum that makes sense for radio production.

Presets

"Studio Room" The default state. Small, warm, present.

"Broadcast Booth" Size 15%, damping 80%, width 40%, wet 8%. Very tight, very subtle. Intimate.

"Open Hall" Size 65%, damping 45%, width 100%, wet 25%. Bigger, more dramatic. For musical jingles and idents.

"Warm Chamber" Size 40%, damping 70%, width 60%, wet 18%. Classic chamber reverb. Flattering on everything.

9. Chorus/Ensemble (**chorus.pd**)

Send bus effect. Thickens and widens sound through modulated delay.

Description text for patch: "Chorus and ensemble effect. Thickens sounds by mixing with a pitch-modulated copy. Makes single oscillators sound like ensembles and pads sound wider. On the send bus — use the send knob on each source module to route signal here."

Implementation: Two delay lines (left and right) with LFOs offset 90° in phase for stereo width. Delay time modulated by LFO around a center point. Use vanilla **delwrite~** / **delread4~** (interpolating read) with **osc~** LFO.

Default State

Parameter	Value
Rate	0.6 Hz
Depth	45%
Delay time (center)	18 ms
Feedback	10%
Width	90%
Mix	50%

Control Ranges

Control	Min	Max	Default	Display
Rate	0.1 Hz	3 Hz	0.6 Hz	Hz
Depth	0%	100%	45%	Percent
Delay time	5 ms	30 ms	18 ms	ms
Feedback	0%	40%	10%	Percent
Width	0%	100%	90%	Percent
Mix	0%	100%	50%	Percent

Presets

"**Ensemble**" The default state. Lush, wide, warm.

"**Gentle Drift**" Rate 0.2 Hz, depth 25%, delay 22 ms, feedback 5%. Barely perceptible widening.

"**Thick Swirl**" Rate 1.2 Hz, depth 70%, delay 12 ms, feedback 20%. Obvious, psychedelic.

"**Shimmer Wash**" Rate 0.4 Hz, depth 60%, delay 25 ms, feedback 30%, width 100%. Deep, dreamy.

10. Filter — Standalone (**filter.pd**)

Send bus effect. Resonant multimode filter with LFO and envelope follower.

Description text for patch: "Resonant filter with slow sweep modulation. Adds movement to static textures. The envelope follower mode makes the filter respond to input dynamics — louder input opens the filter. On the send bus."

Implementation: Use ELSE (**svfilter~**) (state-variable filter providing simultaneous LP/HP/BP outputs). Selectable output. LFO via (**osc~**). Envelope follower via (**env~**) (vanilla) scaled and applied to cutoff.

Default State

Parameter	Value
Mode	Low-pass
Cutoff	1800 Hz
Resonance	30%
LFO rate	0.08 Hz
LFO depth	50%
Envelope follower	0%
Mix	50%

Control Ranges

Control	Min	Max	Default	Display
Mode	—	—	LP	Radio: LP / HP / BP
Cutoff	40 Hz	10000 Hz	1800 Hz	Hz
Resonance	0%	85%	30%	Percent
LFO rate	0.01 Hz	5 Hz	0.08 Hz	Hz
LFO depth	0%	100%	50%	Percent
Envelope follower	0%	100%	0%	Percent
Mix	0%	100%	50%	Percent

Presets

"Slow Sweep" The default state. Gentle, slow LP sweep.

"Underwater" LP, cutoff 400 Hz, no LFO, resonance 40%. Static low-pass. Everything sounds submerged.

"Radio Dial" BP, cutoff 1000 Hz, resonance 50%, LFO rate 2 Hz, depth 20%. Nasal, scanning-through-frequencies sound.

"Wah Pulse" LP, envelope follower 70%, resonance 45%, no LFO. Filter responds to input dynamics.

"High Pass Rise" HP mode, cutoff 100 Hz, LFO rate 0.05 Hz, depth 80%. Bass slowly disappears and returns. Tension builder.

11. Tremolo/Gate (`tremolo-gate.pd`)

Send bus effect. Amplitude modulation from smooth tremolo to hard rhythmic gating.

Description text for patch: "Amplitude modulation with variable shape — from smooth vintage tremolo to hard rhythmic gating. The shape control morphs between sine (smooth) and square (choppy). Stereo phase offset creates ping-pong effects. Can sync to the sequencer tempo."

Implementation: LFO (variable shape from sine to square) multiplied against signal amplitude. Shape morphing: crossfade between `osc~` (sine) and a squared version of the same signal. Stereo: two LFOs with controllable phase offset.

Shape morphing detail: At 0% shape, use pure sine. At 100%, use hard square. Intermediate values crossfade or use `pow` to progressively sharpen the sine's peaks and flatten its troughs. A simple approach: `clip~` the sine with increasingly tight bounds, then rescale.

Sync implementation: When sync is enabled, the LFO rate locks to a division of the sequencer's tempo. The sequencer must send a clock or tempo message that this module can receive.

Default State

Parameter	Value
Rate	4 Hz
Sync	Free
Shape	20%
Depth	55%
Phase offset (L/R)	0°
Mix	50%

Control Ranges

Control	Min	Max	Default	Display
Rate (free mode)	1 Hz	12 Hz	4 Hz	Hz
Rate (sync mode)	—	—	1/8	Radio: 1/4, 1/8, 1/16, 1/8T
Sync	—	—	Free	Radio: Free / Sequencer
Shape	0%	100%	20%	Percent (0=sine, 100=square)
Depth	0%	100%	55%	Percent
Phase offset	0°	180°	0°	Degrees
Mix	0%	100%	50%	Percent

Presets

- "**Classic Tremolo**" The default state. Smooth sine, moderate rate, mono.
- "**Stereo Pulse**" Rate 3 Hz, shape 30%, depth 70%, phase offset 180°. Sound bounces left-right.
- "**Hard Gate**" Shape 100%, sync to 1/8 notes, depth 100%. Full rhythmic chopping.
- "**Slow Breathe**" Rate 1 Hz, shape 0%, depth 40%. Very gentle pulsing for long beds.

"Choppy Swing" Sync to 1/8 triplet, shape 60%, depth 80%, phase offset 90°. Bouncing triplet rhythm.

Master Console (`console.pd`)

The main patch. Always open during a session. Everything connects through here.

Layout (top to bottom)

Header Area

- Station name (text label, configurable)
- Toolkit version (text label)
- Current date (text label)

Source Bay

Four source module slots arranged in a row. Each slot contains:

Element	Type	Default
Module name	Text label	"Empty"
On/Off	Toggle	Off
Level	Slider: -inf to 0 dB	-6 dB
Pan	Slider: L to R (0-127 range, center = 64)	Center (64)
Send level	Slider: 0% to 100%	0%
Open module GUI	Bang button	—
Module selector	(when empty, choose which module to load)	—

When a source module is toggled on, its audio feeds the insert bus at the set level and pan. Its send level determines how much signal goes to the parallel effects bus.

Effects Section — Inline Chain

Horizontal display of the inline effects:

Module	Bypass toggle	Open GUI button
Tape Warmth	Default: OFF (active)	Bang
Space	Default: OFF (active)	Bang

Effects Section — Send Bus

Horizontal display of the send effects:

Module	Bypass toggle	Return level	Open GUI button
Chorus	Default: ON (bypassed)	Slider: -inf to 0 dB, default -6 dB	Bang
Filter	Default: ON (bypassed)	Slider: -inf to 0 dB, default -6 dB	Bang
Tremolo/Gate	Default: ON (bypassed)	Slider: -inf to 0 dB, default -6 dB	Bang

Note: send effects default to bypassed, unlike inline effects. The creative opts into them by setting a source's send level above 0% and disabling the bypass.

Sequencer Section

Compact display:

Element	Type	Default
Status	Text label	"Stopped"
Tempo	Number display	92 BPM
Current step	Number display	—
Target module	Text label	—
Open sequencer GUI	Bang button	—

MIDI Section

Element	Type	Default
Input status	Text label	"No MIDI" / "Connected"
Target module	Radio selector	(first active source)
MIDI channel	Number: 1-16	1

MIDI routing: Incoming MIDI note messages on the selected channel are converted to the same note+gate format the sequencer uses, then routed to the target module. This means source modules don't need to know whether they're being driven by the sequencer or MIDI — the interface is identical.

Master Section

Element	Type	Default
VU meter (stereo)	Pd vu objects, L and R	Active, showing signal
Master level	Slider: -inf to +6 dB	-3 dB
Limiter threshold	Fixed	-0.5 dBFS
Limiter active indicator	Toggle (read-only)	—

Limiter implementation: A simple brickwall limiter. Use clip~ -1 1 after applying gain, but with a slightly lower ceiling ($-0.5\text{ dBFS} \approx 0.944$). When the limiter engages, the indicator lights up. Under normal operation with the master at -3 dB, the limiter should never trigger.

Recording Section

Visually distinct area (use a colored canvas background).

Element	Type	Default
Asset type	Radio buttons	Other
Filename	Text field (editable)	Auto: <code>[type]-[YYYY-MM-DD]-[NNN].wav</code>
Output directory	Text display + change button	<code>/export/raw/</code>
Record button	Large toggle, turns red when active	Off
Recording timer	Text display	00:00.0
Last recorded file	Text display	(none)

Asset type options: Jingle, Sweeper, Stinger, Bumper, Bed, Station ID Bed, PSA Bed, Promo Bed, Other.

Recording implementation: Use vanilla `writesf~` to write stereo WAV (16-bit or 24-bit, 44100 Hz). When recording starts, open the file with the auto-generated filename. When recording stops, close the file and write a line to a manifest file (`export/raw/manifest.txt`):

```
filename.wav|asset_type|YYYY-MM-DD HH:MM:SS
```

The post-session script reads this manifest for tagging.

Filename auto-generation: On each new recording, scan `export/raw/` for existing files matching today's date and the selected asset type, and increment the counter. Example sequence: `sweeper-2026-02-21-001.wav`, `sweeper-2026-02-21-002.wav`, etc.

Transport Bar

Bottom of the console.

Element	Type	Function
Global Play	Bang	Starts all active modules and sequencer
Global Stop	Bang	Stops all modules and sequencer
Panic	Bang	Sends note-off to all modules, kills stuck notes/drones

Post-Session Script (`scripts/post-session.sh`)

A bash script that processes the recorded WAV files after a creative session. Run from the terminal.

Usage

```
bash

# Standard usage (normalize + convert + tag)
./scripts/post-session.sh

# Skip normalization (for experienced users who gain-staged manually)
./scripts/post-session.sh --skip-normalize

# Custom station name for ID3 tags
./scripts/post-session.sh --station "Delta Community Radio"
```

Pipeline

The script processes all WAV files listed in `export/raw/manifest.txt`.

Step 1: Loudness Normalization

For each WAV file (unless `--skip-normalize`):

```
bash

# Two-pass loudnorm with ffmpeg
# Pass 1: analyze
ffmpeg -i input.wav -af loudnorm=I=-16:TP=-1:LRA=11:print_format=json -f null - 2>&1 | grep -A 20 "Parsed_loudnorm"

# Pass 2: normalize using measured values
ffmpeg -i input.wav -af loudnorm=I=-16:TP=-1:LRA=11:measured_I=$measured_I:measured_TP=$measured_TP:measured_
```

Target: **-16 LUFS** integrated loudness, **-1 dBTP** true peak ceiling.

Step 2: MP3 Conversion

```
bash

ffmpeg -i normalized.wav -codec:a libmp3lame -b:a 320k -ar 44100 output.mp3
```

320 kbps CBR. Preserves quality for broadcast elements. The original WAV remains in `export/raw/`.

Step 3: ID3 Tagging

Read each line from `manifest.txt`. Parse filename and asset type. Apply tags:

```
bash

ffmpeg -i output.mp3 -metadata title="$TITLE" -metadata artist="$STATION_NAME" -metadata album="Station Audio Elements"
```

Tag	Value
Title	Filename without extension
Artist	Station name (configurable, default "Station")
Album	"Station Audio Elements"
Genre	Asset type from manifest (Jingle, Sweeper, Bed, etc.)
Comment	"Generated YYYY-MM-DD via Station Audio Toolkit"
Year	Current year

The **Genre** field carrying the asset type is critical: LibreTime's smart blocks can filter by genre, enabling automatic playlist construction (e.g., a smart block that pulls all sweepers).

Step 4: File Organization

Move the tagged MP3 into the appropriate subdirectory under `export/`:

Asset Type	Directory
Jingle	<code>export/jingles/</code>
Sweeper	<code>export/sweepers/</code>
Stinger	<code>export/stingers/</code>
Bumper	<code>export/bumpers/</code>
Bed	<code>export/beds/</code>
Station ID Bed	<code>export/station-id/</code>
PSA Bed	<code>export/psa/</code>
Promo Bed	<code>export/promos/</code>
Other	<code>export/other/</code>

The entire `export/` directory (minus `raw/`) is ready to drag into LibreTime's upload interface.

README Content

The toolkit README should include:

Setup Instructions

1. Install Pure Data vanilla (0.54 or later) from <https://puredata.info>
2. Open Pd. Go to Help > Find Externals. Search "else". Click Install.
3. Clone or download this toolkit to any location.
4. Open `console.pd`. This is the main workspace.

Quick Start

1. In the Source Bay, click a module selector and choose "Analog Oscillator Bank."
2. Toggle the module ON. You should immediately hear a warm, detuned sawtooth tone.
3. The sound passes through Tape Warmth and Space by default — everything already has the station sound.
4. Try the presets: click "Warm Bass," "Bright Lead," etc.
5. Open the Sequencer. Click Run. The default pattern drives the oscillator.
6. When you hear something you like, set the Asset Type in the Recording Section, then click Record.
7. When done, click Record again to stop. Your WAV is saved.
8. After your session, run `./scripts/post-session.sh` to normalize, convert to MP3, and tag for LibreTime.

Session Tips

- Start with a Chord Pad on "Warm Bed" as a foundation, then layer other elements on top.
- Use the send knob on source modules to add chorus, filter sweep, or tremolo.
- The sequencer defaults to pentatonic minor — experiment freely, everything will sound musical.
- If connecting a MIDI keyboard, set the MIDI target in the console to whichever module you want to play.
- Record short takes. It's easier to select the best from many options than to try to get one perfect take.

Troubleshooting

- **No sound:** Check that a source module is toggled ON and its level is above -inf. Check that the master level is up. Check Pd's audio settings (Media > Audio Settings) — the correct output device must be

selected.

- **Clicking or popping:** Increase the audio buffer size in Pd's audio settings. 64 or 128 samples is often too low — try 256 or 512.
 - **ELSE objects not found:** Make sure the ELSE library is installed via Deken. Restart Pd after installing.
 - **MIDI not responding:** Check Pd's MIDI settings (Media > MIDI Settings). Select your MIDI device. Ensure the MIDI channel in the console matches your controller's output channel.
-

Implementation Notes for Developer

Pd Patch File Format

Pd patch files are plain text. Each line is a directive:

- `#N canvas X Y W H FONTSIZE;` — creates a canvas/window
- `#X obj X Y objectname args;` — places an object
- `#X msg X Y message;` — places a message box
- `#X floatatom X Y WIDTH LOWER UPPER LABEL_POS LABEL RECEIVE SEND;` — number box
- `#X symbolatom X Y WIDTH LOWER UPPER LABEL_POS LABEL RECEIVE SEND;` — symbol box
- `#X connect FROM_OBJ FROM_OUTLET TO_OBJ TO_INLET;` — connects two objects

Object indices are zero-based in order of creation within the canvas.

Layout Guidance

- Object positions (X, Y) are in pixels from top-left of the canvas.
- Arrange controls in logical groups with ~20px spacing between objects.
- Place labels (text comments via `#X text X Y label text;`) above or to the left of controls.
- Use `#X obj X Y cnv SIZE WIDTH HEIGHT RECEIVE SEND LABEL XOFF YOFF FONTSIZE BGCOLOR LABELCOLOR;` for colored background panels to visually group sections.
- GUI sliders: `#X obj X Y hsl WIDTH HEIGHT LOWER UPPER LOG INIT SEND RECEIVE LABEL XOFF YOFF FONTSIZE BGCOLOR FGColor LABELCOLOR DEFAULT STEADY;`
- GUI toggles: `#X obj X Y tgl SIZE INIT SEND RECEIVE LABEL XOFF YOFF FONTSIZE BGCOLOR FGColor LABELCOLOR DEFAULT_VALUE;`
- GUI radio buttons: `#X obj X Y hradio SIZE NEW_OLD INIT CELLS SEND RECEIVE LABEL XOFF YOFF FONTSIZE BGCOLOR FGColor LABELCOLOR DEFAULT_CELL;`

- GUI VU meter: `#X obj X Y vu WIDTH HEIGHT RECEIVE LABEL XOFF YOFF FONTSIZE BGCOLOR LABELCOLOR SCALE;`
- GUI bang: `#X obj X Y bng SIZE HOLD INTERRUPT INIT SEND RECEIVE LABEL XOFF YOFF FONTSIZE BGCOLOR FGColor LABELCOLOR;`

Inter-Module Communication

Use Pd's `send` and `receive` objects for communication between patches. Namespace convention:

Send/Receive Name	Purpose
<code>tk-insert-L</code> , <code>tk-insert-R</code>	Stereo audio from source modules to insert bus
<code>tk-send-L</code> , <code>tk-send-R</code>	Stereo audio from source modules to send bus
<code>tk-master-L</code> , <code>tk-master-R</code>	Stereo audio to master output/recording
<code>tk-note</code>	Note number from sequencer/MIDI to active source
<code>tk-gate</code>	Gate (0/1) from sequencer/MIDI to active source
<code>tk-velocity</code>	Velocity/accent from sequencer/MIDI
<code>tk-tempo</code>	BPM from sequencer (for sync features)
<code>tk-clock</code>	Clock tick from sequencer (for tempo sync)
<code>tk-source[N]-level</code>	Per-source level from console (N = 1-4)
<code>tk-source[N]-pan</code>	Per-source pan from console
<code>tk-source[N]-send</code>	Per-source send amount from console
<code>tk-record-start</code>	Trigger to start recording
<code>tk-record-stop</code>	Trigger to stop recording
<code>tk-panic</code>	All-notes-off message

Prefix all send/receive names with `tk-` to avoid collisions with any other Pd patches the user might have open.

Building and Testing Order

Recommended implementation order:

1. **Tape Warmth** — simplest effect, tests the inline signal chain concept

2. **Analog Oscillator Bank** — core source module, tests audio generation and control layout
3. **Console (basic)** — source bay + inline effects + master output + recording. Test the signal flow.
4. **Sequencer** — tests control routing to source modules
5. **Remaining source modules** (Noise Sculptor, FM Voice, Chord Pad, Sample Player)
6. **Remaining effects** (Space, Chorus, Filter, Tremolo/Gate) — add to console send bus
7. **MIDI input** — add to console
8. **Presets** — implement preset recall for all modules
9. **Console (complete)** — all sections, transport bar, asset type selector, filename generation
10. **Post-session script**
11. **README**

Known Pd GUI Limitations

- Pd's GUI toolkit (Tk) is spartan. Controls will be functional but not polished. This is expected.
 - Slider and knob positions are approximate — manual adjustment of X/Y coordinates may be needed after initial generation to achieve clean visual layout.
 - There are no true "knob" objects in vanilla Pd. Use horizontal or vertical sliders instead.
 - Canvas objects can serve as colored backgrounds for grouping, but they cannot have rounded corners or complex styling.
 - The creative should expect a synthesis environment, not a DAW interface. The labels, grouping, and preset system bridge the usability gap.
-

Summary of Key Design Principles

1. **Sound on first open.** Every source module makes a good, broadcast-ready sound the moment it's toggled on. Defaults are opinionated, not zeroed out.
2. **Sweet-spot ranges.** Control ranges are constrained to where useful sounds live. The creative can't accidentally dial in silence or harsh noise within the default slider travel.
3. **Station sound by default.** Tape Warmth and Space are inline and active. Everything passes through the warmth chain before it reaches the output.
4. **Scale safety.** The sequencer defaults to pentatonic minor, making it nearly impossible to produce dissonant patterns.
5. **Named presets.** Presets are labeled for what they're useful for ("Warm Bed," "Rising Sweep") not what they are technically ("BP 400Hz LFO 0.05").

6. **Record-ready workflow.** Asset type tagging, automatic filenames, and the post-session pipeline mean the creative's output goes from the toolkit to LibreTime with minimal friction.
7. **Minimal dependencies.** Pd vanilla + ELSE. That's it. Five-minute setup on any platform.